

Malware command-and-control traffic analysis

Follow-along: find the flow that should not be there, then stop it

Overview

Software that has been installed on a machine without its owner's consent is not much use to whoever put it there until it can be reached. Reaching it means the machine opens a connection outwards, on a schedule, to a host the operator controls, and asks whether there is anything to do. That request is a **check-in**, the program that sends it is an **implant**, and the host it talks to is its **controller**. Almost everything else an intrusion does is occasional. The check-ins are not: they carry on at their own rate whether or not anybody is at the keyboard, and that regularity is what makes them findable in a capture that holds thousands of other packets.

Definition: Implant, controller, check-in

An **implant** is the program running on a compromised machine. Its **controller** is the host it takes instructions from. A **check-in** is one request from the implant to the controller, sent whether or not there is anything to report; the reply is whatever the operator has queued, and most of the time there is nothing. This lab's implant sends one check-in every thirty seconds and does nothing else. It steals no files, opens no shell, and touches no other machine on the network, because the exercise is finding a flow and stopping it, not the rest of an intrusion.

Finding it is harder than the description makes it sound, because a working network is already full of programs that talk to a fixed address on a fixed schedule. Every machine in this lab polls a monitoring endpoint every thirty seconds, which is exactly the implant's interval, so a search for regularity alone returns several flows and most of them are supposed to be there. You will separate them on three other measurements, and then write an egress policy on the gateway that stops the check-ins while everything the network is supposed to be doing carries on working. The policy is what you are graded on, and it has to survive two changes to the incident: the operator alters how the implant checks in, and then moves the controller. Neither change is announced, and you find out what happened the same way you found the flow in the first place.

This lab works from one vantage point, the gateway, and one kind of evidence, the packets crossing it. It does not cover how the implant got onto the machine, how to remove it, or what host-based evidence would say; a process list on the infected workstation would answer the first question of Part 2 in one line, and that is a different exercise. It also uses no name resolution at all: every destination here is an address, so nothing in this network produces the DNS records that a real investigation would read alongside the flows.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

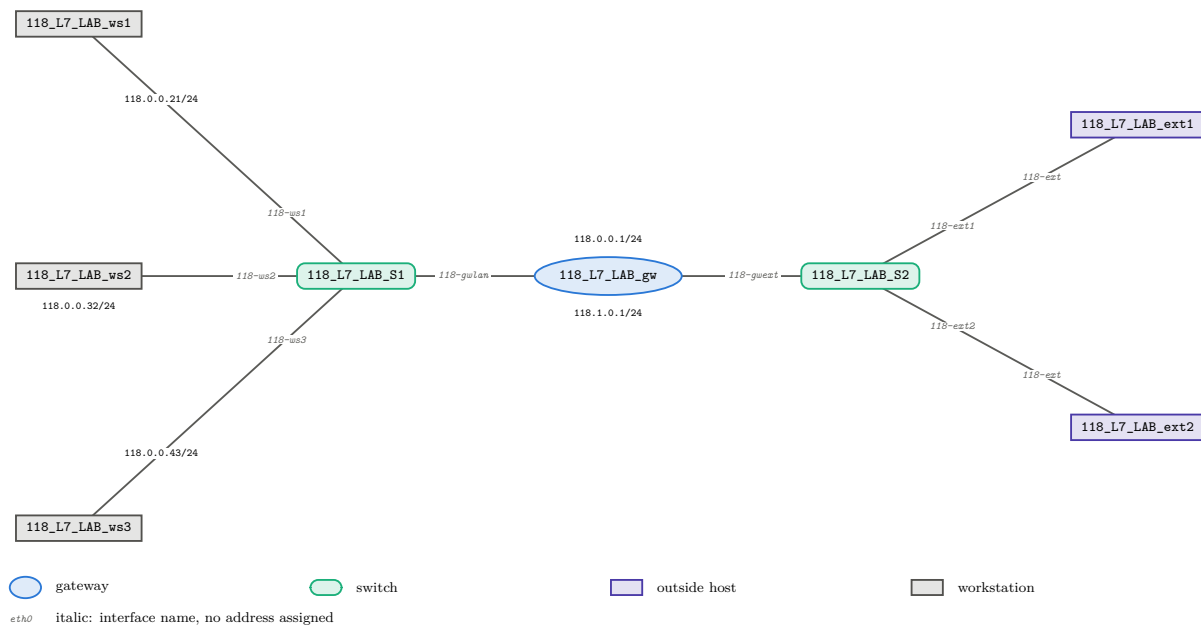
If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

The root menu lists one category per row. Select **Application layer attacks and mitigations**, then **Malware command-and-control traffic analysis**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its images, which takes a few minutes; later spawns take seconds. The workstations start producing traffic as soon as they are configured, so give the lab a minute before your first capture.
- **Status** prints the lab's state and its success oracle: what the gateway is configured to do, how much the controller has heard recently, and whether the legitimate services still answer. The last two stay closed until the gateway has a policy, because until then there is nothing to report on and Parts 1 and 2 are measured from your own capture. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does. The whole of this lab is worked on **gw**, the gateway; the other five are there for afterwards.
- **Advance the incident** moves the incident to its next stage. Parts 4 and 5 tell you when to use it. It says which stage you are now at and nothing else, because working out what changed is the exercise.
- **Reset to baseline** throws away the policy you wrote on the gateway, puts the incident back to its first stage, and empties the controller's record, without tearing the lab down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup



Eight containers on two segments. Three workstations and the gateway’s inside interface sit on one switch; the gateway’s outside interface and two hosts standing in for the internet sit on another. No port of one switch is a port of the other, so the only path from a workstation to anything outside runs through the gateway.

Two properties of the gateway make this lab possible, and both are stated here because every measurement you take depends on them. It forwards packets between the segments, and it does not translate addresses. Without translation, a packet arriving at the outside interface still carries the workstation’s own source address, so a capture taken there says which of the three machines produced each flow. Behind a translating gateway every flow would appear to come from 118.1.0.1 and Part 2 would have no answer.

Container	Role	Address	Listening on
118_L7_LAB_ws1	workstation	118.0.0.21	nothing
118_L7_LAB_ws2	workstation	118.0.0.32	nothing
118_L7_LAB_ws3	workstation	118.0.0.43	nothing
118_L7_LAB_gw	gateway, inside side	118.0.0.1	nothing
	gateway, outside side	118.1.0.1	
118_L7_LAB_ext1	outside host	on 118.1.0.0/24	tcp/80
118_L7_LAB_ext2	outside host	on 118.1.0.0/24	tcp/80 and more
118_L7_LAB_S1, S2	switches	none	none

The outside addresses are missing from that table on purpose, and so are the services behind them. Between them the two outside containers hold five addresses; which of those are in use, what each one serves, and which one is the controller is what Part 1 and Part 2 establish from a capture. The three workstations are set up identically and run the same programs, apart from the one extra thing running on one of them.

The tools you need are on the gateway: `tcpdump` to record traffic, `tshark` to read the recording back, `nft` to write the policy, and `awk` and `sort` for the arithmetic. `curl` is on every machine, and `vi` and `nano` are too.

Assessment

This handout carries **eighteen** graded questions, numbered in order and placed at the step that produces the evidence for them. Answer them in a separate document. Most of them ask for a number or a list you measured; record what you got, not what you expected, and if the two differ say so. The answers that identify the implant are worth almost nothing on their own, because the lab's own scripts contain them and reading those would take a minute: what is graded is the measurement each identification rests on, which no script will hand over.

Part 1: what leaves this network

An investigation that starts by looking for something suspicious finds whatever the analyst already suspected. Start instead by writing down everything, and only then ask which entries are strange. For traffic leaving a network, everything means: which outside addresses were contacted, by how many inside machines each, how often, and how much data went each way.

Open a shell on the gateway and record two minutes of traffic on the outside interface. The lab has been generating traffic since it was spawned, so there is no need to make anything happen first.

```
# 118_L7_LAB_gw: the gateway, where the whole lab is worked
tcpdump -i ext -w /tmp/inventory.pcap -n &
sleep 120
kill %1
```

Definition: Capture file

A file holding whole packets exactly as they crossed an interface, headers and payload, with a timestamp on each. `tcpdump -w` writes one; `tshark -r` reads one back. Recording to a file rather than reading the packets as they arrive is what makes the measurements below possible: a capture can be filtered, counted and re-read as many times as the question needs, and the same file settles a question you did not think to ask while it was being taken. The `-n` keeps `tcpdump` from turning addresses into names, which here would mean nothing, since this network runs no name service.

While that runs, note what it does not see. The gateway's outside interface carries traffic between the two segments and nothing else, so anything a workstation sends to another workstation never appears in this file. That is a real limit on what any conclusion below can claim.

Definition: Conversation

In `tshark`, one conversation is all the traffic between a pair of endpoints, and for TCP an endpoint is an address and a port together. Two connections from the same machine to the same server therefore count as two conversations, because the client's port differs. `-z conv,tcp` prints one row per conversation with a frame and byte count in each direction, when it started, and how long it lasted.

Read the file back as a table of conversations:

```
# 118_L7_LAB_gw
tshark -r /tmp/inventory.pcap -q -z conv,tcp
```

That table has one row per connection and is too long to read as an inventory. Collapse it to a

count of connections per destination. A connection starts with one TCP segment that has the SYN flag set and the ACK flag clear, so counting those counts connections, whatever the service on the far end turns out to be:

```
# 118_L7_LAB_gw: connections per destination address and port
SYN='tcp.flags.syn==1 && tcp.flags.ack==0'
tshark -r /tmp/inventory.pcap -Y "$SYN" -T fields -e ip.dst -e tcp.dstport \
  | sort | uniq -c | sort -rn
```

Definition: Display filter and field extraction

`-Y` takes a display filter: an expression over the fields `tshark` has dissected out of each packet, which selects the packets to work on. `-T fields -e <name>` then prints only the named fields, tab separated, one line per packet, instead of a human-readable summary. Together they turn a capture into columns that `sort`, `uniq` and `awk` can count.

Now the second column of the inventory, which is the one that does most of the work later: how many distinct inside machines contacted each destination. Print the destination and source of every connection start, keep one line per distinct pair, and count the pairs per destination:

```
# 118_L7_LAB_gw: distinct inside hosts per destination
tshark -r /tmp/inventory.pcap -Y "$SYN" -T fields -e ip.dst -e ip.src \
  | sort -u | cut -f1 | uniq -c
```

Question 1. Give the inventory as a table: every outside address contacted during your capture, the port, the number of connections, and the number of distinct inside hosts that contacted it. Four destinations should appear.

Question 2. One of those destinations differs from the other three on the host count. State which, and explain why a destination contacted by one machine out of three is worth a second look, given that plenty of legitimate destinations in a real network are used by a single machine.

Question 3. Pick one conversation with the busiest destination and one with the destination from question 2, and give the byte count in each direction for both, from the `-z conv,tcp` table. State which direction carries more in each case, and say what a client fetching a web page is expected to look like on that measurement.

Part 2: telling one periodic flow from another

Four destinations, and the host count has already made one of them interesting. That is not enough to act on: a rule written on a hunch that costs a legitimate service its connectivity is worse than no rule. This part measures three properties that separate a check-in from an ordinary periodic request, and the point of measuring three is that no single one of them is conclusive.

Take a longer capture. Three minutes gives five or six intervals to average over, and two gaps are not a period.

```
# 118_L7_LAB_gw
tcpdump -i ext -w /tmp/measure.pcap -n &
sleep 180
kill %1
```

How regularly does it repeat

The gap between one connection and the next is the difference between their timestamps. `frame.time_relative` is seconds since the first packet in the file, which is easier to subtract than a wall-clock time, and `awk` does the subtraction:

```
# 118_L7_LAB_gw: gaps between connections from one host to one destination
SYN='tcp.flags.syn==1 && tcp.flags.ack==0'
tshark -r /tmp/measure.pcap -T fields -e frame.time_relative \
  -Y "$SYN && ip.src==118.0.0.21 && ip.dst==118.1.0.10" \
  | awk 'NR>1 { printf "%.2f\n", $1-p } { p=$1 }'
```

The filter names a source as well as a destination, and that is not decoration. Three machines polling the same endpoint every thirty seconds, each starting at a different moment, produce arrivals at that endpoint every few seconds in a repeating but uneven pattern. Group by destination alone and a set of regular flows looks like one irregular one.

Question 4. Run that gap calculation for each source and destination pair you found in Part 1, and report the gaps. Name every pair whose gaps are constant to within a second, and give the period of each.

Question 5. Now run it against the monitoring endpoint by destination only, with no `ip.src` in the filter, and give the gaps. Explain the difference between that list and the per-pair lists from question 4, in terms of what the three pollers are each doing.

How big are the request and the reply, every time

A page fetch varies in size because pages vary in size. A check-in does not: the implant sends the same fields every time and the controller replies with the same few bytes whenever there is nothing queued. The lab's traffic is plain HTTP at this stage, so both lengths are dissected fields:

```
# 118_L7_LAB_gw: the length of every request body, and of every reply body
tshark -r /tmp/measure.pcap -Y 'http.request.method=="POST"' -T fields \
  -e ip.src -e ip.dst -e http.request.uri -e http.content_length
tshark -r /tmp/measure.pcap -Y 'http.response' -T fields -e ip.src \
  -e http.content_length | sort | uniq -c
```

Question 6. Give the request body length and the reply body length for the flow you identified in question 2, and the range of reply lengths for each of the other three destinations. State which destination's replies never change length.

Question 7. The check-ins in your capture all carry the same request target and the same `User-Agent` string as every other HTTP request in this network. Explain what that costs an analyst who was planning to write a detection rule on either field, and name the property from questions 4 to 6 that does not have that weakness.

Naming the two machines

Question 8. State which inside address is running the implant and which outside address and port its controller is on. List the measurements that support each of the two answers, and for each measurement say what a legitimate flow with the same property would look like. A rule that would also fire on the monitoring poll is not an answer.

Part 3: stopping the check-ins without stopping anything else

The gateway is the only path between the two segments, so it is where a packet can be stopped. What you write there has to do two things at once, and the second is the one that is easy to lose: the check-ins stop, and the machines keep working. A gateway that forwards nothing satisfies the first condition perfectly.

Definition: nftables table, chain, hook and policy

nftables is the Linux packet filter. Rules live in a **chain**, chains live in a **table**, and a table names an address family: **inet** covers IPv4 and IPv6 together. A chain becomes a **base chain** when it is given a **hook**, which is the point in the kernel's packet path where it runs. **forward** is the hook for packets the machine is routing between two of its interfaces, which is every packet in this lab. Only a base chain carries a **policy**, and the policy is what happens to a packet that no rule in the chain accepted or dropped: **accept**, which is the default, or **drop**.

The two spellings of a policy are the whole of the difference between this part and Part 5. A chain whose policy is **accept** blocks the traffic its rules name and passes everything else, so `ip daddr 203.0.113.9 drop` in such a chain stops that one destination and nothing more. A chain whose policy is **drop** passes only the traffic its rules name.

Create the table and the chain, then write one rule. Decide what to key the rule on before you type it: you measured several properties of the check-in flow in Part 2, and the rule can be written against any of them.

```
# 118_L7_LAB_gw
nft add table inet egress
nft "add chain inet egress forward \
    { type filter hook forward priority filter ; policy accept ; }"
```

The chain is named **forward** after its hook. Do not name it **fwd**: that is a keyword in the **nft** language, and every later command mentioning the chain would fail to parse.

Definition: Counter

counter in a rule makes the kernel keep a packet and byte tally for that rule, printed by `nft list table`. It changes nothing about what the rule does. It is how you tell a rule that is working from a rule that has never matched anything, and those look identical without it.

Write your rule with a **counter** in it, then read the table back:

```
# 118_L7_LAB_gw: one rule of your own, then read the table
nft add rule inet egress forward <your match here> counter drop
nft list table inet egress
```

Question 9. Before you run the Status action, predict what it will report for the controller's three windows (last 60, 300 and 900 seconds) two minutes after your rule is in place, and say why the three numbers should not all be the same at that moment.

Wait two minutes, then run the Status action. The wait is not padding: at this stage the implant checks in every thirty seconds, and the shortest window Status reports looks back over the last sixty, so a check-in that arrived just before your rule did is still inside it for a minute afterwards. It reports how many check-ins reached the controller in each of three windows, how long ago the last one arrived, and whether the three legitimate services still reply to a workstation. Those two halves together are what containment means here.

Question 10. Give the four numbers Status reports for the controller and the three status codes it reports for the legitimate services. Then give the packet count on your drop rule from `nft list table inet egress`.

Question 11. Your rule's counter keeps rising. Say what that tells you about the state of the infected machine, and describe what the same counter would do if the implant had stopped running instead.

Question 12. A colleague proposes dropping everything from the infected workstation's address until the machine can be rebuilt. Compare that with the rule you wrote: name one thing it does that yours does not, and one cost it carries that yours does not. Say which you would apply on a Friday afternoon and why.

Part 4: the implant changes what it looks like

Nothing you did stopped the implant. It is still running, and its operator can change what it does. Use the **Advance the incident** action once. It tells you which stage the incident has reached and nothing else; work out what changed the way you worked out anything else here, by capturing and reading.

```
# 118_L7_LAB_gw: after advancing the incident
tcpdump -i ext -w /tmp/stage2.pcap -n &
sleep 180
kill %1
tshark -r /tmp/stage2.pcap -q -z conv,tcp
```

Your Part 3 rule either held or it did not, and which one depends entirely on what you keyed it on. Run the Status action before you read any further: the result means more before you know the reason for it.

Definition: TLS record and Client Hello

A TLS connection carries its data in **records**, each with a short unencrypted header giving the record's type and length, followed by the encrypted bytes. The first message of a connection is the **Client Hello**, which is sent before anything is encrypted and is therefore readable in a capture: it offers the versions and ciphers the client supports, and carries extensions, one of which is `server_name`, the name the client is trying to reach. In TLS 1.3 the server's certificate is sent after the keys are established, so a capture holds no readable copy of it.

Three things are worth extracting from the new capture. What the Client Hello offers, whether a certificate is visible at all, and how long the encrypted records are:

```
# 118_L7_LAB_gw
tshark -r /tmp/stage2.pcap -Y 'tls.handshake.type==1' -T fields \
-e ip.dst -e tcp.dstport -e tls.handshake.extensions_server_name
tshark -r /tmp/stage2.pcap -Y 'tls.handshake.type==2' -T fields \
-e tls.handshake.extensions.supported_version
tshark -r /tmp/stage2.pcap -Y 'tls.handshake.type==11' | wc -l
tshark -r /tmp/stage2.pcap -Y 'tls.record.opaque_type==23' -T fields \
-e ip.src -e ip.dst -e tls.record.length
```

The first line reads the Client Hello, the second reads the version the server chose out of the Server Hello, the third counts certificate messages, and the fourth lists the length of every record carrying application data. A frame holding two records prints both lengths separated by a comma. Note that `tls.record.opaque_type` is the field to filter on rather than `tls.record.content_type`: in TLS 1.3 an encrypted record declares itself as application data whatever it actually holds, and Wireshark gives that outer declaration its own field name.

Question 13. State what changed about the check-in flow between your Part 2 capture and this one: destination port, protocol, and the gaps between connections. Give the smallest and largest gap you measured. Then say whether your Part 3 rule held, and explain the outcome from what you keyed the rule on.

Question 14. Give the number of certificate messages your capture holds and the version from the Server Hello, and give the `server_name` the Client Hello carries. Explain the last of those three results from what the implant was configured with, and say what the field would have held if the implant had been pointed at a name instead.

Question 15. Compare the byte counts in each direction for a check-in conversation in this capture with the ones you recorded in question 3, and give both. The direction that carries more has changed. Explain why, and state what that costs the measurement you used in question 3 as a general signal.

Two things did not change. The check-in goes to the same address it went to before, and the length of the record carrying it is the same in every connection: the plaintext inside it is the same length every time, the cipher this connection negotiated adds a fixed number of bytes to every record it protects, and nothing here pads a record to disguise its size.

Part 5: naming what is allowed instead of what is not

A rule that names the controller's address holds through everything Part 4 did. It holds because the address did not change, which is a property of the incident rather than of the rule. Advance the incident once more and give it a minute.

Definition: Blocklist and allowlist

A **blocklist** names what is forbidden and permits the rest; an **allowlist** names what is permitted and forbids the rest. In `nftables` the difference is the base chain's policy, not the rules: the same rule that drops one destination under a policy of `accept` would be one entry in a blocklist, and a chain with a policy of `drop` and three `accept` rules is an allowlist of three destinations. A blocklist is only as good as the enumeration behind it, and nobody can enumerate the addresses an operator has not used yet.

Question 16. Before you run the Status action, predict whether the rule you have been running since Part 3 still stops the check-ins, and give the reason. Then run it and report what happened.

The allowlist is three destinations, because Part 1 found three legitimate ones. Two rules before them make it work at all.

Definition: Connection tracking

The kernel keeps a table of the connections it has seen and gives each packet a state: **new** for one that starts a connection, **established** for one belonging to a connection already in the table, **related** for one belonging to a connection a tracked connection spawned. **ct state** matches on it. It matters at the **forward** hook because that hook sees traffic in both directions: a rule that permits a workstation's request to a server says nothing about the server's reply coming back, and under a policy of **drop** the reply is discarded and the request never completes.

Build the policy in one go. Deleting the table first is deliberate: rules are evaluated in the order they were added, so appending an **accept** to a chain that already holds a **drop** matching the same packets adds a rule that never runs.

```
# 118_L7_LAB_gw
nft delete table inet egress
nft add table inet egress
nft "add chain inet egress forward \
    { type filter hook forward priority filter ; policy drop ; }"
nft add rule inet egress forward ct state established,related counter accept
nft add rule inet egress forward ip saddr 118.0.0.0/24 \
    ip daddr { <the three addresses> } tcp dport 80 counter accept
```

Run Status again once two minutes have passed. All three legitimate services should reply, and the controller should hear nothing. One thing that worked a minute ago no longer does: a workstation can no longer ping anything outside, because nothing in the chain permits ICMP and the policy is now **drop**. That is what default-deny means, and it is the reason it is expensive to operate.

Question 17. Give the controller's four numbers and the three service status codes from Status, and the packet counts on both of your **accept** rules. Then say what the counter on the **established,related** rule would read if you had left that rule out, and what a workstation would see.

Question 18. The implant is still running and still trying. Name two things this policy costs the people running the network that the Part 3 rule did not, and one thing an operator could do next that this policy would still stop. Say which of the two policies you would leave in place for a year and why.

Checks: what "done" means

The Status action is the oracle, and it stays closed until the gateway has a policy, because until then there is nothing for it to report on and Parts 1 and 2 are measured from your own capture. Once there is one, it reports:

- **What the controller received**, over the last 60, 300 and 900 seconds, and how long ago the last check-in arrived. Before any policy is in place the last two windows fill at two check-ins a minute. A policy that works empties the 60-second window first and leaves the 900-second one full for a quarter of an hour.

- **What still works, from ws1:** the three legitimate services, each replying 200 or not replying at all. 000 is `curl` giving up with no reply, which is what a dropped packet looks like from a client.
- **Where the lab stands:** four marks. The controller has heard nothing for at least a minute; all three services still reply; the chain's policy is **drop**; and the whole of that is still true at stage 3, with the controller moved.

The first two marks together are the containment result. Either one alone is easy to get: a gateway that forwards nothing scores the first, and a gateway with no policy scores the second.

Safety note

The lab runs on two Open vSwitch bridges inside containers on your own machine, with no route off the host. The implant, the controller and the traffic between them exist only inside those containers, and the implant sends one HTTP request on a timer to an address the lab created. The capture, the filter and the policy commands are the ordinary tools of network operations, and here they are aimed at containers this lab spawned and nothing else.

Teardown

Remove the lab when you are finished: use the **Teardown** action in the TUI, or run `scripts/teardown.sh` directly. It removes this lab's eight containers and nothing else.